

FGV EMAp

João Pedro Jerônimo e Eduardo Adame

Aprendizado de Máquina

Revisão para A3

Rio de Janeiro

2026

Conteúdo

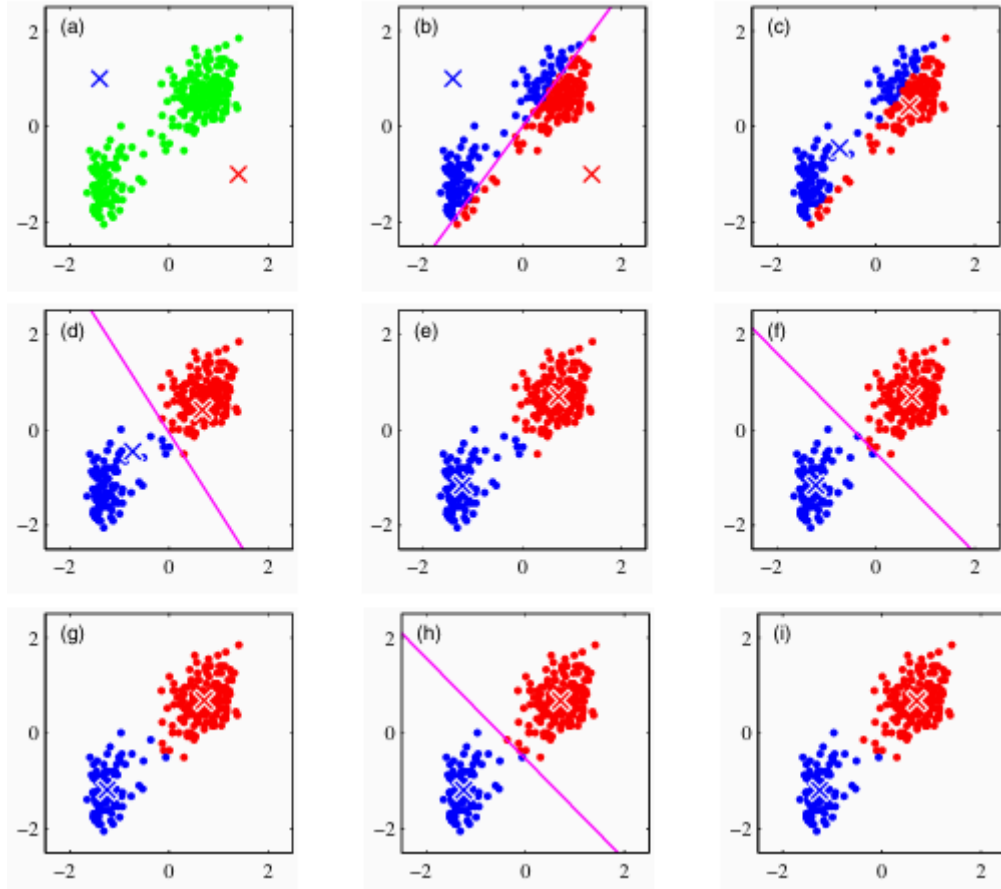
- 1 K-means 3
 - 1.1 Introdução 4
 - 1.2 O Algoritmo 4
- 2 Gaussian Mixture Models 7
- 3 Variational Autoencoders 9
- 4 Generative Adversarial Networks 11
- Referências 13

K-means

1.1 Introdução

A partir desse momento, vamos começar a estudar métodos de machine learning não supervisionados. Diferente do aprendizado supervisionado, onde temos um conjunto de dados rotulado, no aprendizado não supervisionado, os dados não possuem rótulos e o objetivo é encontrar padrões ou estruturas subjacentes nos dados.

O primeiro que vamos estudar é um dos algoritmos de machine learning não supervisionado mais simples que existe, o K-means. Ele é um algoritmo de clustering que busca particionar os dados em K grupos distintos com base em suas características.



1.2 O Algoritmo

Considere o problema de classificar pontos em um espaço. Naturalmente, pensamos que os pontos mais próximos um do outro devem pertencer ao mesmo grupo. O K-means é um algoritmo que busca encontrar esses grupos de forma iterativa, ajustando os centróides dos clusters até que a convergência seja alcançada.

Suponha que temos um dataset $D = \{x_1, \dots, x_N\}$ com pontos aleatórios de um espaço euclidiano de dimensão D . O nosso objetivo é particionar o dataset em um conjunto de K clusters, onde, no momento, vamos supor que K é conhecido. Também definimos um conjunto de K centróides $\mu_1, \dots, \mu_K$. Nosso objetivo então é associar cada um dos pontos x_i a um dos centróides μ_k , de forma que a soma das distâncias quadradas entre os pontos e seus centróides seja minimizada. Definimos também uma variável de associação $r_{nk} \in \{0, 1\}$, que indica se o ponto x_n pertence ao cluster k ou não. Então podemos definir nossa função de custo, também conhecida como *função de distorção*, como:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2 \quad (1)$$

que é a soma das distâncias quadradas entre cada ponto e o centróide do cluster ao qual ele pertence. O objetivo do K-means é minimizar essa função de custo.

Teorema 1.2.1 (Valor ótimo de r_{nk}): Para um conjunto fixo de centróides $\mu_1, \dots, \mu_K$, a escolha ótima de r_{nk} é dada por:

$$r_{nk} = \begin{cases} 1 & \text{se } k = \operatorname{argmin}_j \|x_n - \mu_j\|^2 \\ 0 & \text{caso contrário} \end{cases} \quad (2)$$

Demonstração: Perceba que (1) é uma função linear de r_{nk} . Os termos envolvendo diferentes n são independentes entre si, então podemos otimizar para cada n separadamente de forma que $r_{nk} = 1$ para qualquer valor k que minimize $\|x_n - \mu_k\|^2$ e $r_{nk} = 0$ para todos os outros valores de k . Ou seja, simplesmente assinalamos cada ponto ao cluster cujo centróide está mais próximo (o que pode ser expresso como $k = \operatorname{argmin}_j \|x_n - \mu_j\|^2$). $\square$

Teorema 1.2.2 (Valor ótimo de μ_k): Para um conjunto fixo de associações r_{nk} , a escolha ótima de μ_k é dada por:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}} \quad (3)$$

Demonstração: Perceba que (1) é uma função quadrática de μ_k . Para encontrar o valor ótimo de μ_k , podemos derivar a função de custo em relação a μ_k e igualar a zero:

$$\frac{\partial J}{\partial \mu_k} = 2 \sum_{n=1}^N r_{nk} (\mu_k - x_n) = 0 \quad (4)$$

Rearranjando os termos, obtemos:

$$\mu_k \sum_{n=1}^N r_{nk} = \sum_{n=1}^N r_{nk} x_n \quad (5)$$

Dividindo ambos os lados por $\sum_{n=1}^N r_{nk}$, obtemos a expressão desejada para μ_k . $\square$

Perceba que no Teorema 1.2.2, o centróide μ_k é simplesmente a média de todos os pontos que pertencem ao cluster k . Isso faz sentido intuitivamente, pois queremos que o centróide represente o “centro” do cluster.

Perceba, porém, que o (1) não é uma função convexa de r_{nk} e μ_k juntos, então não podemos otimizar ambos ao mesmo tempo. O K-means resolve isso alternando entre otimizar r_{nk} e μ_k iterativamente até que a convergência seja alcançada.

Além disso, o K-means é sensível à inicialização dos centróides. Diferentes inicializações podem levar a diferentes soluções locais, então é comum executar o algoritmo várias vezes com diferentes inicializações e escolher a melhor solução encontrada.

Note que eu mostrei uma forma de aplicar o K-means em batch, ou seja, considerando todos os pontos de uma vez. Existe também uma versão estocástica online do K-means, onde os centróides são atualizados à medida que novos pontos chegam. Não entraremos em detalhes da derivação, mas esse método se utiliza do procedimento de Robbins-Monro para atualizar os centróides de forma incremental. Esse procedimento é utilizado para resolver equações do tipo

$$\mathbb{E}[Z(\theta)] = 0 \quad (6)$$

onde não observamos $Z(\theta)$ diretamente, mas sim uma amostra $Z_{n(\theta)}$ ruidosa de $Z(\theta)$. Em Machine Learning Pattern and Recognition[1], Bishop nota que a equação do gradiente de ∇J é exatamente no formato de (6)

$$\sum_{n=1}^N r_{nk}(\mu_k - x_n) = 0 \quad (7)$$

então chegamos no procedimento de atualização estocástico do K-means, que é dado por:

$$\mu_k^{t+1} = \mu_k^t + \eta_n(x_n - \mu_k^t) \quad (8)$$

onde η_n é a taxa de aprendizado, que deve ser escolhida de forma que ela diminua quanto mais pontos forem vistos, para garantir a convergência do algoritmo.

Algoritmo K-means (Batch)

```

    }
1  function K-means(D, K) {
2      initialize  $\mu_1, \dots, \mu_K$  randomly
3      repeat {
4          for  $n = 1$  to  $N$  {
5               $r_{nk} = 0$  for all  $k$ 
6               $r_{nk} = 1$  for  $k = \operatorname{argmin}_j \|x_n - \mu_j\|^2$ 
7          }
8          for  $k = 1$  to  $K$  {
9               $\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}$ 
10         }
11     }
12 until convergence

```

O algoritmo do K-means é baseado na distância euclidiana para medir a discrepância entre os pontos e os centróides. No entanto, essa medida não é adequada em todos os cenários. Por exemplo, se os clusters forem labels? Além de que torna J não resistente à outliers. Para lidar com isso, podemos utilizar o *K-medoids*, que é uma variação do K-means que utiliza uma outra distorção qualquer $\mathcal{V}(x, x')$ em vez da média para calcular os centróides.

$$\tilde{J} = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \mathcal{V}(x_n, \mu_k) \quad (9)$$

Gaussian Mixture Models

Variational Autoencoders

Generative Adversarial Networks

Referências

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.